The Y Language

A Formal Definition

the y compiler project

September 2026

Abstract

This document is the language definition of Y, a small statically-checked systems language that compiles directly to Linux x86-64 executables. It records the complete surface language, its type system, and its evaluation semantics — with particular precision for the mutating array operators (`+=`, `*=`), the increment/decrement and compound-assignment families, and their evaluated-exactly-once rules, so that these features are specified rather than discovered. Where the language is quirky, the quirk is written down; where a construct is reserved but unimplemented, it says so.

Contents

1 Overview **3**

2 Lexical structure **4**
2.1 Source text . . . 4
2.2 Comments . . . 4
2.3 Identifiers and keywords . . . 4
2.4 Literals . . . 4
2.5 Operators and punctuation . . . 5

3 Syntax **6**
3.1 Structure and modules . . . 6
3.2 Packages . . . 6
3.3 Declarations . . . 7
3.4 Types . . . 8
3.5 Statements . . . 8
3.6 Expressions . . . 9

4 Types and static semantics **10**
4.1 Scalar types . . . 10
4.2 Arrays . . . 10
4.3 Records . . . 10
4.4 Unions . . . 11
4.5 Type aliases . . . 12
4.6 Maps . . . 12
4.7 Options and pointers . . . 12
4.8 Void . . . 12
4.9 Functions . . . 13
4.10 Annotations and inference . . . 13
4.11 Conversions . . . 13

4.12 Operator typing . . . 14
4.13 Compound assignment typing . . . 14
4.14 The poison law . . . 14

5 Expressions and evaluation **14**
5.1 Numeric semantics . . . 14
5.2 Bitwise operators . . . 15
5.3 Logic . . . 15
5.4 Strings . . . 15
5.5 Assignment . . . 15
5.6 Increment, decrement and compound assignment . . . 16

6 Arrays **16**
6.1 Structure: length and capacity . . . 17
6.2 Operators . . . 17
6.3 Aliasing and visibility . . . 18
6.4 Fixed arrays . . . 18
6.5 Slices . . . 18
6.6 Byte strings . . . 18
6.7 Operators are the interface . . . 19

7 Records and maps **19**
7.1 The value model . . . 19
7.2 Records . . . 19
7.3 Maps . . . 20
7.4 Equality . . . 20
7.5 Copying . . . 20

8 Functions, closures, options, pointers **21**
8.1 Functions . . . 21
8.2 Tail calls . . . 21
8.3 Closures . . . 21
8.4 Options . . . 22
8.5 Void . . . 22
8.6 Pointers . . . 22

9 Builtins **22**
9.1 Sequences and text . . . 22
9.2 String methods . . . 23
9.3 Conversions . . . 24
9.4 Format specifications . . . 24
9.5 Binary data . . . 25
9.6 Options . . . 25
9.7 Input/output and the environment . . . 25
9.8 Files (the fs module) . . . 26
9.9 The web (the http, tls and net modules) . . . 27
9.10 Cryptography (the crypto and x509 modules) . . . 28
9.11 Pointers . . . 28

10 Errors and diagnostics **29**
10.1 Compile time . . . 29
10.2 Warnings . . . 29
10.3 Run time . . . 30
10.4 Exit status . . . 30
10.5 Reproducibility . . . 30

1 Overview

Y is a small imperative language with a static checker, designed so that a complete implementation of the language fits in one head: the compiler is roughly seventeen thousand lines of Y itself (`src/y/`, plus the runtime library it splices into programs) and compiles a program to a *static* x86-64 ELF executable using only its own assembler and ELF writer — no system assembler, linker, or C toolchain is involved at run time or at build time of user programs.

Implementations

Two implementations exist and are kept in exact step:

- **the reference compiler**, written in Y (`src/y/`), self-hosting and byte-reproducible;
- **the C copy**, a line-for-line port of the same compiler in C (`src/c/`), with its own value model (`src/c/yv.c`). The two must produce byte-identical executables for every input.

This document defines the language both implement. Implementation detail is deliberately excluded except where it *is* observable behavior (value sharing, integer wraparound, trap conditions).

Notation

Grammar productions are given in EBNF, with `typewriter` for terminals, *italic* for nonterminals, `[ ]` for option, `{ }` for repetition, `|` for alternation, and `( )` for grouping. Semantic rules are written as equational desugars or prose; the desugars are normative: an implementation may use any mechanism that produces the same observable behavior.

A taste

```
// values, references, and the mutating operators
let xs = [1, 2, 3];      // heap array: len 3
xs += 4;                 // in-place push:        [1, 2, 3, 4]
xs *= 0;                 // in-place prepend:     [0, 1, 2, 3, 4]
let ys = xs + [9];       // NEW array, xs untouched

// the target of a compound assignment runs exactly once
let a = [1, 2];
def idx() -> long { print("called"); 0 };
a[idx()] += 10;          // idx() runs ONE time

// records are reference values too
let r = { name: "y", nums: [1, 2] };
r.nums += 3;             // visible through every binding of r
```

2 Lexical structure

2.1 Source text

A program is a sequence of bytes, conventionally UTF-8. The lexer works on bytes; only ASCII is significant to the grammar. Both `"\n"` (LF) and `"\r\n"` (CRLF) end a line. Positions reported in diagnostics are 1-based line and column, the column counted in bytes.

2.2 Comments

```
// a line comment runs to the end of the line
/* a block comment
   does not nest: the first */ closes it */
```

Comments are whitespace: they separate tokens and nothing more.

2.3 Identifiers and keywords

`isalpha`(c) is true for `A`–`Z`, `a`–`z` and `_`; `isdigit`(c) for `0`–`9`. An identifier is

$$\textit{identifier} ::= \textit{isalpha}\ \{\ \textit{isalnum}\ \}$$

so `_` is a word character: `is_digit` is a single identifier, not three tokens. The prefix `y_` is *reserved*: identifiers beginning with `y_` are used by the compiler's generated code and must not be written by hand.

The keywords are:

`let` `const` `def` `if` `else` `while` `for` `foreach` `in` `return` `true` `false`

2.4 Literals

Integers. Decimal, binary (`0b1010`) and hexadecimal (`0x1F`) radix literals are supported. Any of them may group digits with `_`, which is ignored: a radix literal (`0x` / `0b`, either case) in any position after the prefix (`0xFF_FF`, `0b1010_1010`), a decimal one anywhere after its first digit (`1_000_000`; a leading `_` starts an identifier instead). A prefix needs a digit of its radix: `0x`, `0b_` and `0b2` are errors (`binary literal '0b2' has no digits`), one for the whole word. There is no octal form: `017` is decimal seventeen. All integer literals are *long* (64-bit). A radix literal spells the 64 bits themselves, so a mask may set the top bit: `0xFFFF_FFFF_FFFF_FFFF` is `-1` and `0x8000_0000_0000_0000` the smallest *long*; digits past 64 bits wrap away (modulo 2^{64}).

Floats. A digit sequence with a decimal point and/or an exponent: `1.5`, `1.5e+3`, `2.0e-1`. `_` may group digits after the first digit of the whole part, of the fraction and of the exponent: `1_000.5`, `0.000_001`, `1e1_0` (`1._5` and `1e_5` are parse errors). A dot followed by a non-digit is *not* a fractional part — `a..b` lexes as `a`, `..`, `b`. Float literals are IEEE-754 binary64 (*float*).

Strings. Double-quoted byte strings:

escape	meaning
`\n` `\t` `\r`	LF, HT, CR
`"\"` `"\""` `"\'"`	backslash, quote
`\xHH`	byte with hex value `HH`
`\c` (any other *c*)	*c*; the backslash is dropped

An escape is read as a single byte, and an unrecognised `\c` simply drops the backslash (`"q\q"` is `"qq"`), so a NUL is spelled `"\x00"` — which is not `"\0"` (that is the character `0`). A `str` is a length-prefixed byte string, so an escaped NUL inside one is ordinary data and `len` counts it (use `[byte]` arrays, Section 9.5, for bulk binary data).

Byte strings. `b"..."` and `b'...'` are *byte string literals*: a `[byte]` value spelled like a Python `bytes` literal. The escapes are the string escapes above, so binary data writes naturally: `b"\x7fELF"` is the four-byte ELF magic. Either quote spells the same type; single quotes exist only for byte strings.

Format strings. `f"..."` is a string literal with *fields*, as in Python:

```
f"{name} is {age} ({share:.1%})"
```

A field `{e}` becomes `tostring(e)`, and `{e:spec}` becomes `e` formatted as *spec* says (Section 9.4). The text around the fields takes the string escapes, and `{{` and `}}` (or `\{` and `\}`) are literal braces. An f-string is sugar for concatenation: `f"a{x}b"` means `"a" + tostring(x) + "b"`, a *str* like any other.

A field holds one expression — calls, string literals, record literals and nested f-strings included — and may span lines; an error inside it points at its own line and column. The expression ends at the first `}` or `:` outside brackets, except a `:` that answers a `?`: `{ok ? a : b}` works, while a cons has to be parenthesised, `{(x :: xs)}`. The spec is plain text up to the field's `}`; it cannot hold another field.

Booleans and none. `true`, `false`. The empty option is spelled `none`, a predeclared name rather than a keyword (Section 4.7). There is no `null`: the word is an ordinary (undefined) identifier.

2.5 Operators and punctuation

kind	tokens
three- and four-character	`>>> <<= >>= >>>=`
two-character	`== != <= >= && || :: .. -> => ++ - << >>` `+= -= *= /= %= &= |= ^=`
single-character	`+ - * / % < > = ! & . ? | ^ ~`
punctuation	`( ) { } [ ] , ; :`

Maximal munch. The lexer always takes the longest operator that matches, exactly like C:

- `x+++y` is `x ++ + y`, not `x + ++y`;
- `x-y` is `x- y` (a parse error), not `x - -y`; write `x - -y` when negation is meant;
- `/=` is one token; `//` starts a comment, and `/ == y` must be written with a space to divide;
- `x>>>=1` is the compound `x >>>= 1`;

Contextual keywords. `none`, `match`, `type` and `void` are ordinary identifiers to the lexer: `none` is a predeclared name (Section 4.7), `match` is the match-expression keyword only when an expression and `{` follow it, and `type` starts a type alias only when followed by `name =`. Each stays usable as an ordinary name elsewhere.

3 Syntax

3.1 Structure and modules

A *program* is a sequence of *statement*s executed in order. Source files may be combined with imports; an import splices the imported file's text in front of the importing file's own code, under a namespace alias:

```
import util from "./util.y";    // util.f(...) addresses defs of util.y
import fs;                      // a module by name: fs.read_file(...)
```

An `export { a, b }` clause, when present, lists what a file offers to importers; its list may run over several lines. It is *by convention* the last statement of a file, but nothing enforces that: an import splices the imported file's text and drops every `export` clause, so a clause anywhere in the file has the same effect. (A file's defs are spliced whole; namespaces are resolved by rewriting, not by the export list.) Paths resolve relative to the importing file's directory. A missing module is a fatal error. Without a path, `import fs;` finds a module by name: `fs.y` next to the importing file first, then the standard library's `std/fs.y`. An import takes a line of its own: everything after `import` on that line names the module, so a second statement on the same line becomes part of the name.

A module imported by name keeps its own scope. Its top-level `def`s and `let`s are reached as `fs.read_file`, never bare, so they cannot collide with the program's names: a program may define its own `exists`, or `let cwd = fs.cwd();`. Inside the module its own names are bare, and they come before the program's. A binding named like the module hides it where it is in scope: after `let fs = {st: 1};`, `fs.st` is that record's field. Its type aliases are its own the same way: the program names them `m.T` (`let p: geo.Pt = geo.origin();`, a pattern `geo.Pt p => ...`) and may declare a `Pt` of its own, which the module's code never sees.

The files imported by path share one namespace with the program: `util.f` is rewritten to plain `f`, and the alias only says which file a name comes from. A later top-level `def` replaces an earlier one of the same name. Within one file that is deliberate (the last definition wins). Across files it would silently hand one file's callers another file's function, so a `def` that replaces one from another file is an error, reported at the later definition and naming the other (`duplicate definition 'helper': m.y:4 defines it too`). The standard-library code the compiler splices in itself is the exception: a program's own `readline` replaces the prelude's. Nested `def`s and top-level `let`s follow the usual scoping, as if the files were one text.

A file's bytes can be compiled in as a string: on a line of its own,

```
embed HELP from "help.txt";    // const HELP = "<the file's bytes>";
```

declares the `const` with the file's contents, read when the program is compiled. The path resolves like an import's, a missing file is a fatal error, and line numbers after the declaration are unchanged.

3.2 Packages

A *project* is a directory with a manifest, `y.toml`, and `ycc build [DIR]` builds it (`DIR` defaults to the current one):

```
[package]
name = "app"                  # required: an identifier, no keyword
lib = "src/lib.y"             # optional: what importers of app get
src = ["src"]                 # optional: where bare imports also look

[bin]                         # the programs: build/app from
app = "src/main.y"            #   src/main.y, build/tool from
tool = "src/tool.y"           #   src/tool.y
```

```
[dependencies]
json = { path = "../json" }
http = { git = "https://example.com/http.git", rev = "v1" }
```

The manifest is a subset of TOML: one `[table]` header or `key = value` per line, a value being a string (`"..."` with `\\`, `\"`, `\n` and `\t` escapes, or `'...'` as written), an array of strings, or an inline table of strings; `#` starts a comment. `[package]` may also carry `version`, `description`, `license` and `authors`, which the build ignores; any other key or table is an error. A bin may not be named `obj` or `git` (`build/obj` keeps the package objects, `build/git` the clones). Paths are relative to the manifest's directory.

A dependency is another project, named by its own manifest exactly as the key names it, that has a `lib`. Its `lib` file and the files that file imports form a *package*, compiled on its own: in a bin, or in a package's `lib`, `import json;` (on a line of its own) makes the top-level `def`s and `let`s of the `json` package available as `json.parse(s)`, `json.limit`. Such names stay qualified: a bin's own `parse` is another function. Parameter defaults and optional parameters work across the boundary; a package's globals are read-only to its importers (`cannot assign to constant 'json.limit'`), and its type aliases are its own (an importer spells the type out). A project's own `lib`, when it has one, is a package its bins import the same way. A bare import names a dependency before any file: `import json;` in a project depending on `json` is the package even next to a `json.y`; otherwise it looks next to the importing file, then in the `src` roots of the importing file's project, then in the standard library.

Each bin is linked with the packages it imports, directly or through other packages, and each package's top-level code runs once, before its importers' (dependencies first). A package may depend on others; a dependency cycle is an error, and so is one name coming from two different places — unless the project itself declares that name, which settles it for every package. `y_` names are global across packages (they are the runtime's override hooks).

`build/obj` keeps each package's compiled object. A package compiles again only when its text, the paths it was read from, the compiler, or the interface of a package it imports (the names, types and defaults it exports) changed; a change elsewhere in a dependency leaves its importers' objects as they are. `ycc build -g` links the same objects with line information. A package's warnings (Section 10.2) print when it compiles, and `-w` drops them.

A git dependency is cloned into `build/git` — its `rev` (a tag, branch or commit), or the default branch without one — and pinned in `y.lock` to the commit it resolved to; later builds use that commit until its `y.lock` entry is deleted. `ycc vendor [DIR]` copies the project's git dependencies into `vendor/`, which then wins over the network. Git runs through `/bin/sh` with the compiler's environment.

3.3 Declarations

declaration ::= `let` *name* [`:` *type*] `=` *expr* `;`
| `const` *name* [`:` *type*] `=` *expr* `;`
| `def` *name* `(` *params* `)` [`->` *type*] *block*
| `type` *name* `=` *type* `;` (alias)

params is a comma-separated list of *name* `:` *type*, each optionally followed by a literal default (`name: T = literal`). A parameter with a default may leave its type out: it is the literal's (`punct = "!"` is a `str`). Every parameter has a type — one with neither is an error, `parameter 'x' has no type (annotate it)`. Return types are optional (inferred from the body, Section 4.10). A `const` name cannot be rebound: `=`, the compound assignments and `++`/`--` on it are rejected (`cannot assign to constant 'n'`, with the operator in parentheses

for the compound forms). The binding is constant, not the value: a `const` array or record is still a reference, so `a[i] = v`, `r.x = v` and the in-place `+=`/`*=` on a `const` array change its contents.

Function literals may also appear bound or anonymous: `let f = def (x: long) -> long { x };`.

3.4 Types

type ::=	`long` | `byte` | `bool` | `float` | `str` | `void`	
	| `[` *type* `]`	(heap array)
	| `[` *type* `;` *nat* `]`	(fixed array)
	| *type* `?`	(option)
	| `*` *type*	(pointer)
	| `(` [*type* {`,` *type*}] `)` `->` *type*	(function)
	| `{` [*field* {`,` *field*}] `}`	(record shape)
	| `{` *type* `=>` *type* `}`	(map)
	| *type* `|` *type*	(union of records)
field ::=	*name* [`?`] `:` *type*	(`?`: optional)

A record type is structural and its field order free (Section 4.3); it lists every field of its records (`{ k: str }` is the record with the one field `k`); a field written `name?: T` is optional, of type `T?`, and a literal may leave it out (Section 4.3). A union's members are record types (Section 4.4). `nat` is an integer literal from 0 to $2^{31}-1$ (2147483647), decimal or radix (`[0; 0x10]`); a float, a larger or negative number or a name there is an error at the size. The type name `int` is accepted as a synonym of `long` in annotations. Any other name must be a `type` alias in scope: `let s: strr = "x"` is rejected as an unknown type.

3.5 Statements

statement ::=	*declaration*
	| *expr* `;`
	| *assignment* `;`
	| `return` [*expr*] `;`
	| *block*
	| `if (` *expr* `)` *block* [`else` (*block* | *statement*)]
	| `while (` *expr* `)` *block*
	| `for (` [*stmt*] `;` [*expr*] `;` [*stmt*] `)` *block*
	| `for` *name* `in` *expr* `..` *expr* *block*
	| `foreach (` *name* `in` *expr* `)` *block*
	| *block*

block is `{` *statement** [*expr*] `}`: a block is an *expression* whose value is its trailing expression (if any). An empty block `{}` is legal and void; a trailing `;` makes the block void. Statements are separated by `;`, which may be left out only after a statement that ends in `}` (an `if`, loop, `def` or block) or before the `}` closing the block: `f() g()` is an error, and so is any token the grammar does not expect — nothing is skipped silently. `foreach` iterates the elements of an array; the `for` range form iterates `lo..hi` with the low bound inclusive and the high bound exclusive. A C-style `for` without a condition loops until a `return` leaves it, as in C.

3.6 Expressions

primary ::= *long* | *float* | *string* | `true` | `false` | *name*
| `(` *expr* `)` | *block*
| `[` [*expr* {, *expr*}] `]` | `[` *expr* `;` *nat* `]`
| `b` *string* | `c` *character* (byte string / byte)
| `f` *string* (format string)
| `{` [*name* `:` *expr* {, . . .}] `}` `{` [*string* `:` *expr* {, . . .}] `}`
| `def` `(` *params* `)` [`->` *type*] *block*

postfix ::= *primary* { `(` [*args*] `)` | `[` *expr* `]` | `[` *expr* `?:` *expr* `?` `]` | `.` *name* | `!` | `++` | `-` }
unary ::= [`!` | `-` | `~` | `*` | `&` | `++` | `--`] *postfix*

Postfix `e!` unwraps an option: it is exactly `unwrap(e)` (Section 8.4) and, like every postfix form, binds tighter than the prefix operators — `!x!` is `!(x!)`. The two-character token `!=` is always inequality. In a record literal a bare name is shorthand for the field of that name: `{x, y}` is `{x: x, y: y}` (but `{ x }` alone is a block whose value is `x`).

Operator precedence

Binary operators are left-associative (`::` is the exception: it is right-associative, so `1 :: 2 :: a` is `1 :: (2 :: a)`); lower rows bind tighter:

level	operators	notes
1	`||`	short-circuit
2	`&&`	short-circuit
3	`== !=`	structural equality
4	`< > <= >=`	numeric, or lexicographic *str*
5	`::`	one-element cons (Section 6)
6	`|`	bitwise or (Section 5.2)
7	`^`	bitwise exclusive or
8	`&`	bitwise and
9	`<< >> >>>`	shifts
10	`+ -`	numeric, string/array `+`
11	`* / %`	numeric

Unlike C, the bitwise operators bind tighter than the comparisons, so `x & 1 == 0` means `(x & 1) == 0`.

Assignment forms

assignment ::= *place* `=` *expr*
| *place* *compound-op* *expr* `+= -= *= /= %=` `&= \|= ^= <<= >>= >>>=`
| *place* `++` | `++` *place* | *place* `--` | `--` *place*

A *place* is a name, an index expression `a[e]` (arrays and maps), a slice `a[lo:hi]`, or a field expression `o.f` (records), possibly nested: `grid[0][1]`, `rec.v[2]`, `m['k']! [i]` (a map read

is *V?*, so unwrap it before indexing). Field access is symbolic and exclusive: `r.f` for records, `m['k']` for maps. Pointers add `*p = e`. `++`, `--` and the compound forms store back into the place they read, so they take a name, an element or a field: a slice, `*p` or `o!` there is `cannot assign to this expression ('+=')`, with the operator in parentheses. The bytes of a *str* and the `.length` and `.ptr` of an array or a string are read-only to every form. Section 5.6 defines the exact evaluation of every form; all of them evaluate the place's subexpressions **exactly once**.

4 Types and static semantics

Checking is static and *fatal*: a type error is a compile error. The type system is small — scalars, arrays, records, maps, options, pointers, functions and `void` — and mostly structural.

4.1 Scalar types

type	domain	notes
`long`	64-bit signed integers	wraparound on overflow
`byte`	0 . . . 255	stored in one byte inside `[byte]`
`bool`	`true`, `false`	
`float`	IEEE-754 binary64	
`str`	immutable byte strings	length-prefixed; may hold NUL

4.2 Arrays

`[T]` is a heap array of *T*, `[T; N]` a fixed-length array of *T* with *N* a literal. The element type is part of the type and arrays are **homogeneous**: every element of an array has exactly the element type. Heterogeneous array literals are rejected:

```
let bad = [1, "two"];        // error: array element: expected long, got str
```

This applies recursively — `[[1], [2]]` has type `[[long]]`, and every operator that mixes arrays and elements (Section 6) enforces the same rule.

4.3 Records

A record literal `{ a: 1, b: "x" }` has the record type `{ a: long, b: str }`. A record type is the set of its fields: the order they are written in is not part of it, so `{ b: str, a: long }` is the same type (and prints its fields in that order). A field read or write is legal when the static type has the field.

Records are **exact**: a record goes only where its own type is expected, or a union that has it as a member (Section 4.4). A `{ a: long, b: str }` is not a `{ a: long }`: records are shared by reference, their fields at fixed places, so no wider or narrower view of one is sound. There is no record type that takes "any record": `{ k: str }` is the record with exactly one field, `k`.

A record literal written where a record type is declared — an annotated `let`, an argument, a `return` or trailing value, a field, an element — is created as that type: each value is checked against its field's type, converting as any store does (Section 4.11), and a record literal among them likewise. Where a union is declared, the literal is the member with its field names.

A field written `name?: T` is *optional*: its type is `T?`, and such a literal may leave it out (it is then `none`) or give a plain `T`, which is wrapped as `some` would — or a `T?`, or `none`. A record type whose fields are all optional takes `{}`. Where a union is declared, a literal leaving optional fields out is the member it fits; when two fit, the one with exactly its field names, and

otherwise none. The mark is not part of the type: `{ a?: long }` and `{ a: long? }` are one type, and only a literal sees the difference. A literal given to an optional parameter of such a type is created as it too:

```
type Opts = { url: str, timeout?: long, tag?: str };
def get(path: str, opts?: Opts) -> str { ... }
get("/");
get("/", { url: "x" });                 // timeout, tag: none
get("/", { url: "x", timeout: 5 });     // timeout: some(5)
```

4.4 Unions

`A | B` holds a record whose type is exactly one of the members, which are record types (an alias of a union among them adds its members). A record goes into a union when its type is a member; a union goes into another whose members include all of its own. A field that every member has, with one type, is read and written straight through the union; any other needs a `match`, which tells the members apart:

```
type Circle = { name: str, r: float };
type Rect = { name: str, w: float, h: float };
type Shape = Circle | Rect;
def area(s: Shape) -> float {
    match s {
        Circle c => 3.14159 * c.r * c.r,
        Rect { w, h } => w * h
    }
}
let s: Shape = { name: "box", w: 2.0, h: 3.0 };   // a Rect
print(s.name, " ", area(s));                        // box 6.000000
```

In a `match`, a type pattern such as `Circle c` must name a member exactly. A record pattern such as `({name: "box"}) b` matches the members with those fields, and binds the one member with them, or the union of those members. Without `_`, every member needs a type arm. A union prints as the member it holds.

A union value also goes where one of its members is declared — an annotated `let`, an argument, a `return`, a field or an element — or a union of some of its members: that is a *cast*, checked when the program runs. A value holding another member stops the program (Section 10). After the cast the member's own fields read directly:

```
def radius(s: Shape) -> float {
    let c: Circle = s;       // stops the program if s holds a Rect
    c.r
}
```

A union value keeps the fields its members share at one place, whatever member it holds, so reading one is a single load: each member is laid out for the union it is held in. Going between a union and a member, either way, or between two unions, is therefore a cast that may copy the record: a write through the cast value is not seen through the union, nor the other way round, and whether a cast copies depends on the field names, so a program should not count on either. A `match` arm's binding (`Circle c`) is such a cast; a destructuring pattern (`Rect { w, h }`) reads the fields in place. To change a field only some members have, cast, write and store the record back:

```
let r: Rect = s;
r.w = 4.0;
s = r;
```

4.5 Type aliases

`type Name = T;` names a type: the alias stands for *T* wherever it is written (two aliases of one structure are one type). An alias may refer to itself, or to aliases that refer back to it, when the reference sits inside a record's field or a function type:

```
type List = { head: long, tail: List? };
type Expr = Num | Add;
type Num = { n: long };
type Add = { l: Expr, r: Expr };
let e: Expr = { l: { n: 1 }, r: { l: { n: 2 }, r: { n: 3 } } };
```

A cycle with no record field or function type in it, as in `type T = [T];` or `type U = U | { a: long };`, names no type, and is an error at its declaration: `type 'T' refers to itself outside any record field or function type`.

4.6 Maps

`{K => V}` is a dynamic map from hashable keys *K* to one value type *V*. Reading `m[k]` always yields `V?`; see Section 7.3.

K is a hashable key type (`str`, `long`, `byte`, `bool`, `float`), but a **map literal only accepts `str` keys** — the checker's inference for a literal fixes the key type to `str`, so `{ 1: 5 }` infers a shape rather than a map. Build a non-`str`-keyed map from an annotated empty literal and store into it (Section 7.3).

4.7 Options and pointers

`T?` is "a *T* or nothing". Its empty case is `none`; `some`/`unwrap`/`is_some` and the postfix unwrap `x!` (Section 8.4) manipulate it. There is no `null`: `none` is the only empty value, and it exists only at option types — a plain `str` or `long` can never be empty, so `s == none` on a `str` is a type error. On its own `none` has the type "option of anything" (printed `none`) and adopts the option type its context expects. A plain *T* does not widen to `T?`; write `some(v)` — but an optional record field in a literal (Section 4.3) and an optional parameter (Section 8) take a plain *T*. Two options are equal when both are `none` or both hold equal payloads (Section 7.4). `*T` is a raw machine pointer; `&e` yields one (Section 8.6).

4.8 Void

`void` is the type of an expression that computes no value: a loop, an `if` without `else`, a call to a void function (Section 8.5), a bare `return`. Void is not a value: it cannot be bound, passed, printed or returned as one.

Disagreeing `if` arms. When the two arms of an `if`/`else` have different types, the whole `if` is `void` — a statement. That is legal wherever a statement is, and an error wherever a value is needed:

```
if (ok) { 1 } else { "one" };            // fine: a statement
let v = if (ok) { 1 } else { 2 };        // long
let w = if (ok) { 1 } else { "one" };    // error: cannot bind a void value to '
    w'
```

4.9 Functions

`(T, T, ...) -> R`. Every parameter has a type: written, or the type of its default value (`punct = "!"`); a parameter with neither is an error. Return types are inferred when omitted and checked when written. Function values are first-class (they may be bound, passed, stored and called); see Section 8.

4.10 Annotations and inference

Every value has a type the program states or the checker can see; there is no unknown type. Where the type cannot be inferred, annotate:

- `def` parameters are annotated (or have a default): `parameter 'x' has no type (annotate it)`.
- Return types are inferred from the body when omitted: the type of its trailing value, or `void` when it ends in a statement. Written or inferred, every `return` and the body's trailing value must conform, converting as any store does (Section 4.11): `half` below returns `1.0` for `true`, and a `return 1.5` in a body that ends in a `long` is an error.

```
def half(c: bool) { if (c) { return 1; }; 2.5 };
```

- An inferred return type is known once the body has been checked, in program order. A call before that — to a `def` further down that has no `->`, or a value use of it — is an error, `'f' is used before its return type is known (annotate it)`: write the callee's return type. A `def` calling itself needs none. Its body is checked with the call's type unknown, then again as if the type the rest of the body gives were written, so `fact` below is a `long`; a body whose only value is the call itself, like `spin`'s, is an error:

```
def fact(n: long) { if (n < 2) { 1 } else { n * self(n - 1) } };
def spin(n: long) { self(n) };
// error: the return type of 'spin' is not fully known (annotate it)
```

- `let` annotations are optional and checked; an annotated *array literal adopts* the annotated element type: `let b: [byte] = [104, 105];` packs those elements one byte each. Adoption is restricted to literals — assigning an existing `[long]` to a `[byte]` binding is a type error (the layouts differ).
- An empty array literal `[]` and `none` take their type from where they go (an annotated `let`, an argument, an assignment, a comparison). Bound with nothing to say it, the type is not known: `let xs = [];` is an error, `the type of 'xs' is not fully known (annotate it)`; write `let xs: [str] = [];`. An unannotated `{}` is the empty record; under a map annotation it is the empty map: `let m: {str => long} = {};`.

4.11 Conversions

The implicit conversions are: `long` *to* `float` (the only numeric widening), `byte` *to* `long`, `byte` *to* `float`, and the narrowing `long` *to* `byte` at a `byte` position (the value is masked to eight bits: `let b: byte = 258;` stores 2). A `str` also flows into a `*T` position, since a string value is a byte address. A union value converts to one of its members (a cast checked when the program runs, Section 4.4). Apart from these, nothing converts: there is no cast syntax.

A value converts where it is stored under a known type: an annotated `let`; an assignment to a variable, element, field, map entry or pointee; an argument; a `return` or trailing value under the function's return type, declared or inferred; an element of an array literal; and the element that `+=`, `*=` or `::` adds. Array `+` and `+=` with an array operand convert element by element into the left operand's element type: `[0.5] + [7, 8]` is `[0.5, 7.0, 8.0]`. Arithmetic on a `byte` is `long` arithmetic (`-b` is a negative `long`); storing the result back into a `byte` masks it.

Nothing converts a value that is already inside another one. Arrays, maps and records are references and an option is a box, so an existing `[long]` is not a `[float]`, and an existing record `{x: long}` is not a `{x: float}`: a record's numeric field types must match exactly. A record *literal* written where a record type is declared is created as that type, so its values convert: `let p: {x: float} = {x: 1};` stores `1.0`, and an optional field's plain value is wrapped: `let q: {x?: float} = {x: 1};` stores `some(1.0)` (Section 4.3).

4.12 Operator typing

Arithmetic requires numbers; if either side is `float` the result is `float` and the other side widens. `+` additionally allows strings (concatenation, with auto-stringification of the non-string side) and arrays (Section 6); arrays win over strings when both appear. The bitwise operators `& | ^ << >> >>>` and unary `~` take integers only — a `byte` widens — and yield `long`; any other operand is an error naming it (`'&' left operand: expected long, got float`). Comparisons yield `bool`; `<...>=` on two strings is lexicographic byte order. `&&`/`||` accept any values: a non-zero value is true and the result is `bool` (Section 5). Equality works on any two values of the same type; it is by content for scalars and strings, structural for arrays and options (at any depth), and by identity for records and maps (Section 7.4).

4.13 Compound assignment typing

For a target of type T and value of type U:

T	operators	rule
numbers	`+= -= *= /= %=`	U converts to T; stored as T
integers	`&= |= ^= <<= >>= >>>=`	U an integer; a `byte` keeps the low 8 bits
`str`	`+=` only	$U = str$
array	`+=`, `*=` only	Section 6

Anything else is a compile error, e.g. "`arrays support only '+=' and '*=' (in place), not '-='`" or "`'&=' needs an integer target, got float`". `++`/`--` require a numeric target.

4.14 The poison law

Every error is reported once, at its cause. After an error, the expression it was found in gets the *error type* `<error>`, which fits wherever it goes, so it never generates follow-on errors at every enclosing use: in element checks, call arguments and branches (an error arm never makes an `if` void). The error type exists only where an error was reported; a program that compiles has none.

5 Expressions and evaluation

Evaluation is strict and left-to-right; every operand is evaluated exactly once per execution of the enclosing expression. Side effects include calls, reads of places with side-effecting subexpressions, and the mutating array operators.

5.1 Numeric semantics

`long` arithmetic is two's complement modulo 2^{64}: overflow *wraps*, it does not trap. Division and remainder truncate toward zero (C semantics) and are subject to:

- division or remainder by zero is a runtime error (Section 10);

- `LONG_MIN / -1` evaluates to `LONG_MIN` (the quotient does not fit and would fault on x86) and `LONG_MIN % -1` evaluates to `0`. Implementations must special-case these two.

`float` arithmetic is IEEE-754 binary64. Mixed long/float operands widen the long side. Unary minus on a `float` flips the sign, so `-0.0` is a negative zero (`1.0 / -0.0` is $-\infty$) and `-(-x)` is `x` for every `x`.

5.2 Bitwise operators

`&`, `|` and `^` are the and, or and exclusive or of two integers' 64 bits, and `~x` flips every bit of `x` (it is `-x - 1`). A `byte` operand widens first, so `~b` of a `byte` is a negative `long`; the result is always a `long`, and storing it into a `byte` keeps its low 8 bits.

The shifts move the bits of the left operand by the count on the right: `x << n` towards the top, zeros filling in; `x >> n` towards the bottom, the sign bit filling in (arithmetic: `-16 >> 2` is `-4`); and `x >>> n` towards the bottom, zeros filling in (logical: `-16 >>> 60` is `15`). Unlike C, and unlike the x86 instructions, which use only the count's low 6 bits, every count has a meaning:

- a count of 64 or more shifts every bit out: `x << 64` and `x >>> 64` are `0`, and `x >> 64` is `0` or `-1` by the sign of `x`;
- a negative count is a runtime error (Section 10).

The bitwise operators bind tighter than the comparisons (Section 3.6), and a condition still takes a `bool`: `if (flags & 4 != 0)` tests a bit. Radix literals spell masks with the top bit set as they are (Section 2.4).

5.3 Logic

`&&` and `||` short-circuit: the right operand is evaluated only when the left does not decide the result. Operands are not required to be `bool` — any non-zero value is true, and the result of either operator is a `bool`.

5.4 Strings

Strings are immutable values: concatenation `a + b` produces a new string. If exactly one side of `+` is a string, the other side is stringified (Section 9.3) and concatenated. A format string, `f"{x} of {n:>4}"`, writes such a concatenation with fields (Sections 2.4 and 9.4). Comparisons `< <= > >=` on two strings compare bytes lexicographically; `==` compares contents.

Appending. `s += t` means `s = s + t`, but a loop of appends is linear, not quadratic: when `s` is a variable whose address is never taken and that no closure captures, and `s` still holds the string its own previous `+=` built, the bytes of `t` are written in place behind it (the buffer doubles when full). Any other use of `s` — a read, a copy into another binding, an element or argument — ends that ownership, and the next append copies once, so an append never changes a string that anything else can see. Building a string with `+=` costs amortized $O(1)$ per appended byte, like pushing onto an array.

5.5 Assignment

`p = e` evaluates `e` and stores it into the place `p`. Assignment is a statement form, not an operator: it cannot appear inside another expression, so `let y = (x = 7);` and `let y = x = 7;` are syntax errors (so are the compound forms of Section 5.6; only `++` and `--` nest). As the last statement of a block it gives the block its value, the value stored: `let y = { x = 5 };` sets both `x` and `y` to 5, and a function body ending in an assignment returns the assigned value. Section 7 defines what storing into arrays, records and variables means for aliasing.

5.6 Increment, decrement and compound assignment

The four increment/decrement forms and the eleven compound-assignment operators share one rule:

> **Exactly-once rule.** The target place's subexpressions are evaluated *exactly once*. Writing `a[f()] += e` evaluates `f()` one time, no matter how the place is read and written.

Formally, let a *place* be $p ::= x \mid a[i] \mid o.f$, possibly nested. Define $\mathrm{cap}(p)$, the capture of a place into fresh variables (every non-name subexpression runs once):

$$\begin{aligned} \mathrm{cap}(x) &= \epsilon \\ \mathrm{cap}(a[i]) &= \mathrm{cap}(a);\ \text{if } a \text{ is not a name: } \texttt{let } \beta = a\texttt{;}\ \texttt{let } \iota = i\texttt{;} \\ \mathrm{cap}(o.f) &= \mathrm{cap}(o);\ \text{if } o \text{ is not a name: } \texttt{let } \beta = o\texttt{;} \end{aligned}$$

with $\mathrm{read}(p)$ and $\mathrm{write}(p, v)$ the corresponding read and store through the captured names. Then the *numeric* forms desugar as:

$$\begin{aligned} \texttt{p++} &\equiv \mathrm{cap}(p);\ \texttt{\{ let t = read}(p)\texttt{; write}(p, \texttt{t + 1})\texttt{; t \}} \\ \texttt{++p} &\equiv \mathrm{cap}(p);\ \texttt{\{ let t = read}(p)\texttt{ + 1; write}(p, \texttt{t})\texttt{; t \}} \\ \texttt{p-} &\equiv \mathrm{cap}(p);\ \texttt{\{ let t = read}(p)\texttt{; write}(p, \texttt{t - 1})\texttt{; t \}} \\ \texttt{-p} &\equiv \mathrm{cap}(p);\ \texttt{\{ let t = read}(p)\texttt{ - 1; write}(p, \texttt{t})\texttt{; t \}} \\ \texttt{p op= e} &\equiv \mathrm{cap}(p);\ \texttt{\{ let t = read}(p)\ op\ e\texttt{; write}(p, \texttt{t})\texttt{; t \}} \end{aligned}$$

for $op \in \{+, -, \times, \div, \%\}$ on numbers and for the bitwise `& | ^ << >> >>>` on integers (on a `byte` place, `t` is a `byte`: the low 8 bits). The four increment and decrement forms are expressions: postfix yields the *old* value, prefix the *new* one. A compound assignment is, like `=`, a statement form (Section 5.5): it cannot nest inside another expression, and as the last statement of a block it yields the new value of the place (`{ x += 2 }`).

Examples.

```
a[f()]++;          // { let y_t = f(); let v = a[y_t];
                   //   a[y_t] = v + 1; v }   -- f() runs once
rec.v[g()] += 1;   // g() once; the read and the write use the same element
x.y.z += e;        // x and x.y evaluated once
```

Arrays. On array targets `+=` and `*=` are *primitive in-place* operations — they do not desugar through `+` (which copies) — and are defined in Section 6. On string targets only `+=` is legal and concatenates (in place when it can, Section 5.4). Every other combination is a type error (Section 4.13).

Maps and slices. A map store `m[k] = v` evaluates `m`, `k` and `v` once each; a slice assignment `a[lo:hi] = b` the same way (Section 6).

6 Arrays

Arrays are the language's one growable sequence. They are **reference values**: a binding names an array *object*, and mutating operators write into that object so every binding sees the change. Arrays are **homogeneous**: one element type, enforced everywhere.

6.1 Structure: length and capacity

An array object carries two independent quantities:

	meaning	invariant
`len`	number of live elements	`len` ≥ 0
`cap`	elements the storage can hold	`len` $\leq$ `cap`

Indexing is trapped: reading or writing `a[i]` with `i < 0` or `i >= len` is a runtime error ("index out of bounds"). `cap` is never observable through operators — only `len` is (`len(a)`).

Growth policy (amortized doubling).

$$\begin{aligned} \texttt{cap}_0 &= 0 \\ \texttt{cap}_{k+1} &= \max(8,\ 2 \cdot \texttt{cap}_k) \qquad \text{when } \texttt{len} \text{ reaches } \texttt{cap} \end{aligned}$$

That is: the array grows *only* when the capacity is reached; the first growth yields capacity 8 and every later growth doubles it $(8, 16, 32, \ldots)$. Appending n elements one at a time therefore costs $O(n)$ copies total, $O(1)$ amortized per element. The capacity never shrinks.

6.2 Operators

Let a, b be arrays of T, and x a value of type T. The complete operator semantics:

expression	result	effect on operands	value types
`a + b`	new array $= a +\!\!+ b$	none	$T = T$
`a + x`	new array $= a +\!\!+ [x]$	none	type of x is T
`x + a`	new array $= [x] +\!\!+ a$	none	type of x is T
`x :: a`	new array $= [x] +\!\!+ a$	none	x is ONE element
`a += b`	a	appends every element of b, in place	$T = T$
`a += x`	a	pushes x, in place	type of x is T
`a *= x`	a	prepends x, in place	type of x is T
`a = b`	b	aliases: both names name one array	

In prose, matching the design intent exactly:

- `arr1 + arr2` — make a new array which is `arr1` concatenated with `arr2`; the operands are untouched.
- `arr1 += arr2` — append `arr2` to the end of `arr1`, *in place*.
- `arr1 = arr2` — both point to the same array in memory.
- `arr1 + elem` — a new array which is `arr1` concatenated with `[elem]`; only allowed if `elem` has, or converts to, the element type (Section 4.11).
- `elem + arr1` — a new array which is `[elem]` concatenated with `arr1`.
- `+=` means in-place push (of an element) or append (of an array); `+` always means a *new* array; `*=` means prepend the item in place. A push and a prepend both take amortized constant time: an array keeps spare room at both ends and doubles it when it runs out, so building an array from either end is linear.

The one-thing rule. Whether `+=` pushes or appends is decided by the *shape of the value*: an array value appends its elements; any other value pushes as one element. Consequently, to push an array *as one element* of an array-of-arrays, wrap it in a one-element array literal:

```
let rows = [[1], [2]];
rows += [[3]];        // appends the single row [3]
rows += [[4]];        // and the row [4] -- one element each
```

Homogeneity. Every mixing form requires matching element types; the checker rejects mismatches — including the append case, where a non-matching array would splice wrong-typed elements:

```
let rows = [[1], [2]];
rows += [3];   // error: '+=' appended elements: expected [long], got long
```

`a += a`. The right side is captured (its length snapshotted) before appending, so `a += a` doubles `a` with a copy of its original elements.

6.3 Aliasing and visibility

Assigning arrays aliases; mutating operators are visible through every alias:

```
let m1 = [1, 2];
let m2 = m1;      // ONE array, two names
m1 += 3;          // m2 sees it: len(m2) == 3
```

By contrast `+` never mutates: `let ys = xs + [9];` leaves `xs` at its old length.

6.4 Fixed arrays

`[T; N]` arrays are allocated with exactly `N` slots; their length is part of the type, so they reject growth: `+=` and `*=` on a fixed array is a compile error ("cannot grow fixed array `[long; 3]` with `'+='`"). Indexing is bounds-trapped like heap arrays. A fixed array may be sliced; the result is a heap `[T]`.

6.5 Slices

`a[lo:hi]` evaluates to a **new** heap `[T]` holding the elements in [`lo`,`hi`). Missing bounds default to `0` and `len`; bounds are clamped (no trap). All four spellings are legal: `a[:]`, `a[lo:]`, `a[:hi]`, `a[lo:hi]`. Strings slice the same way: `s[lo:hi]` is a new `str` of the bytes in that range, clamped alike. There is no `substr`.

6.6 Byte strings

A `[byte]` array *is* a binary string: `b"..."` literals produce them, `+` concatenates, slices cut, and printing one shows the Python-style repr — printable ASCII verbatim, quote and backslash escaped, `\n \r \t` as escapes, and every other byte as `\xHH`:

```
print(b"\bs x7fELF");     // b'\bs x7fELF'
let hdr = b"AB" + b"\bs x00";
print(hdr);               // b'AB\bs x00'
print(hdr[0:2]);          // b'AB'
```

`\std/struct.y` builds on this with pack/unpack helpers for file headers and binary layouts (`u32be`, `get_i16le`, `pack`, `unpack`, ...).

6.7 Operators are the interface

There are no array mutator functions: `+=` and `*=` *are* push and prepend. `len(a)` is the only builtin whose argument is an array *as such*; `join(xs, sep)` also takes an array but of any element type (each element is stringified), and `bytes/from_bytes` and `fs.write_bytes` take a `[byte]`.

7 Records and maps

Y has *two* key/value types, deliberately distinct:

	record (static)	**map** (dynamic)
type	`{ a: long, b: str }`	`{str => long}`
keys	identifiers, fixed at compile time	any hashable value at run time
values	may differ per field	one homogeneous type V
layout	C struct: fields at fixed offsets	growable key/value store
keying	`r.a`	`m['a']`

7.1 The value model

category	types	assignment
values	`long byte bool float str` options	copies the bits
references	arrays (`[T]`, `[T; N]`), records, maps	**aliases** the object

"Reference" means: every binding of the same object names one storage, and every in-place mutation (`+=`, `*=`, field assignment, map stores) is observable through all of them.

7.2 Records

A record literal `{ name: "y", n: 1 }` allocates one record object of its record type (Section 4.3). At run time a record is a C struct laid out by field name: its fields sorted, the i-th is the word at `+8i` from the record address, whatever order a literal or a type writes them in — so `r.a` compiles to a single load at a compile-time offset. The word before the fields points to a descriptor of the record's type, which a `match` on a union compares to tell the members apart.

```
let r = { v: [7], n: 1 };
let s = r;            // aliases the record
r.v += 8;             // s.v sees it
s.n = 2;              // r.n sees it
```

The shape is fixed: writing a field the shape does not have is a *compile error* (there is no runtime field growth), and a field read requires the field to exist statically.

Record spread. `{a: 0, ...obj}` builds a record whose shape is the *union* of the two shapes — entirely statically; no dynamic shape is involved. Entries merge with last-wins values and first-appearance positions, so `obj` may overwrite `a` and a later explicit key may overwrite what `obj` spread:

```
let b = {x: 1, y: 2};
let c = {y: 9, z: 3};
print({a: 0, ...b});     // {a: 0, x: 1, y: 2}
print({...b, ...c});     // {x: 1, y: 9, z: 3}
print({...b, y: 100});   // {x: 1, y: 100}
```

7.3 Maps

A map type is `{K => V}` where `K` is a hashable key type (`str`, `long`, `byte`, `bool`, `float`) and `V` is one homogeneous value type. Map literals use quoted keys (identifier keys are record syntax):

```
let m = {"a": 1, "b": 2};       // {str => long}
let empty: {str => long} = {}; // an empty map needs the annotation
m["c"] = 3;                     // insert, or overwrite in place
print(m["a"]);                  // 1 -- reads yield V? (here: long?)
print(m["zz"]);                 // none
print(m["a"]! + 1);             // 2 -- x! unwraps
print(m);                       // {"a": 1, "b": 2, "c": 3}
```

Key types. Only `str` keys can be written in a map *literal*: the checker's literal inference fixes the key type to `str`, so `{ 1: 5 }` is rejected as a shape rather than accepted as a `{long => long}`. Every other hashable key type works when the map is built from an annotated empty literal and stored into — which is the only way to write one today:

```
let m: {long => long} = {};    // the annotation is load-bearing here
m[1] = 5;                      // insert
print(m[1]);                   // 5 -- a read is still V? (here: long?)
print(m[2]);                   // none
print(m);                      // {1: 5}
```

`byte`, `bool` and `float` keys behave the same way; see Section 4 for the type-level rule. Keying is symbolic and exclusive: `m['k']` is the map form and is reserved for maps; `r.a` is the record form. Reading `m[k]` yields `V?` — `none` when the key is absent, even though the map's type says `V` — and `m[k] = v` stores in place (the store is visible through every binding of the map). A store to an existing key keeps its position; a new key is appended, so maps print in insertion order. Duplicate keys in a literal collapse the same way (the last value wins, the first position stays). `len(m)` is the number of entries, and `keys(m)` lists the keys in that order (a copy): `foreach (k in keys(m)) { ... }` visits every entry. The map grows as needed (amortized doubling, like arrays). Values are homogeneous: storing a `str` into a `{str => long}` is a compile error, as is a key of the wrong type.

7.4 Equality

`==` and `!=` on two **arrays** are **structural**: same length, pairwise-equal elements. It is never a pointer comparison; `arr == []` is an emptiness test. Each pair of elements is compared with the element type's own `==`, at any depth: numbers as numbers (so `[0.0]` equals an array holding a negative zero), strings by content, arrays and options structurally (`[[1, 2]] == [[1, 2]]` is `true`), records and maps by identity.

Two **options** are equal when both are `none`, or both are `some` with equal payloads, compared with the payload type's own `==`: `some("ab") == some("a" + "b")` and `some([1]) == some([1])` are `true`, `some(1) == some(2)` is `false`. A map read is an option, so `m["a"] == m["b"]` compares the two values (or their absence). `x == none` remains the emptiness test.

On two **records** or two **maps**, `==` compares *identity*: it is true exactly when both names refer to the same object. Two equal-looking literals are two objects, so `{x: 1} == {x: 1}` is `false`; compare fields yourself, or copy the record and compare the copy's fields.

7.5 Copying

There is no implicit deep copy. To duplicate an array use `+` with an empty right side (`a + []`) or slicing (`a[:]`); records can be rebuilt field by field. The language deliberately makes sharing

the default and copying explicit.

8 Functions, closures, options, pointers

8.1 Functions

A `def` declares a function; parameters are typed, the return type is inferred or checked. Write recursion with `self` — that is the portable spelling, and it is what a function *literal* (which has no name) has to use. A *named* `def`'s own name is also in scope inside its body, so `count(n - 1, acc + 1)` and `self(n - 1, ...)` are the same call:

```
def fact(n: long) -> long {
    if (n <= 1) { 1 } else { n * self(n - 1) }
};
```

Functions are first-class values: they may be bound (`let f = g;`), stored in arrays and records, passed to other functions, and called through any of those (`f(1)`, `fs[0](1)`, `r.handler(1)`). Calling a non-function is a compile error.

8.2 Tail calls

A self-call in *tail position* — the body's trailing expression, a `return`'s value, or an if-arm's trailing expression — recycles the stack frame instead of growing it: the arguments are evaluated, the frame is torn down, and control re-enters the function. Tail recursion therefore runs in constant stack space:

```
def sum_to(n: long, acc: long) -> long {
    if (n == 0) { acc } else { self(n - 1, acc + n) }
};
```

Calls outside tail position (`self(x) + self(y)`, `let t = self(x)`, inside loops) are ordinary calls and stack as usual.

8.3 Closures

A function literal captures the enclosing bindings it reads — and only those that are already in scope where the literal is written (a name bound by a `let` *after* the `def` is not in scope and is an undefined-variable error, exactly like reading a local before its declaration). Captured bindings are **shared**: a write inside the closure is visible outside, and a write outside is visible inside.

```
def counter() {
    let n = 0;
    def tick() -> long { n = n + 1; n };
    tick
};
let t = counter();
t(); t();
print(t());        // 3 -- the one shared n
```

Each *invocation* of the enclosing function creates fresh bindings, so closures created in different iterations see different cells ("`let` semantics"):

```
while (i < 3) {
    let x = i;
    fns += def() -> long { x };
    i = i + 1;
};
```

8.4 Options

`T?` is the type of "a *T* or nothing"; its empty case is `none`. `some(v)` boxes a value as present, `is_some(v)` tests, and `unwrap(v)` — or its postfix spelling `v!` — returns the payload and is a *runtime error* on `none` (it is a programmer error, loudly). `x == none` is the is-empty check, and two options are equal when their payloads are (Section 7.4); an option of an array is an optional reference, not a copied one.

```
let line = readline();          // str?
if (line == none) { print("eof"); } else { print(line! + "!"); };
```

8.5 Void

A `def` whose result is not a value is `void`. Write `-> void` to say so: the body's trailing value is then discarded, and `return` takes no value. An unannotated `def` whose body ends in a statement infers `void`, and then every `return` in it must be bare: a `return 5;` there is an error, and so is a bare `return;` in a body that ends in a value. A void call is a statement; using it as a value — binding it, printing it, passing it as an argument — is a compile error:

```
def log(msg: str) -> void { print("log: " + msg); };
log("hi");                  // fine: a statement
let x = log("hi");          // error: cannot bind a void value to 'x'
```

8.6 Pointers

`*T` is a raw address of one *T*; `*p = v` stores, `*p` reads. The address-of operator `&e` is deliberately narrow:

- `&x` — address of a local or parameter;
- `&a[i]` — address of an array element (any array expression: `&grid[0][i]` is legal);
- `&arr` on a whole array — *both* kinds, `[T]` and `[T; N]` — or `&` of any temporary, is a compile error ("cannot take the address of a whole array"). A binding names the array *header*, not its first element, so `&arr` would hand out the header; write `&arr[0]`, which works for either kind.

Pointers are for interfacing with the system-call level (buffers for `read`/`write`-style calls); ordinary code uses arrays and records.

9 Builtins

The following functions are available without declaration. "Trap" marks a runtime error (Section 10).

9.1 Sequences and text

`len(x)` Length of a string (bytes), array (elements) or map (entries) as `long`. Records are rejected (a record's field count is part of its type).

`keys(m)` The keys of map `m` in insertion order, as an array `[K]` of its own (growing it leaves the map alone): `foreach (k in keys(m)) { print(k, m[k]!); };`.

`range(n)` `[0, 1, ..., n-1]`.

`trim(s)` Strip leading/trailing space, tab, CR, LF.

`join(xs, sep)`
: Concatenate `[str]` with a separator.

`split(s, sep)`
: Split on every occurrence of a non-empty separator; returns `[str]` (empty for empty separator).

`index_of(s, sub)`
: First index of `sub` in `s`, or `-1`.

`last_index_of(s, sub)`
: Last index, or `-1`.

`ord(s)`
: First byte as `long`, or `-1` for empty.

`chr(n)`
: One-byte string of `n` (masked to 8 bits).

`byte_at(s, i)`
: Byte at `i` as `long`; traps out of bounds. Cheap per-byte access (no copy).

There is no `substr`: a substring is a slice, `s[i:i + n]` (Section 6.5).

9.2 String methods

A `str` value takes methods: `"str 1 2 3".split(" ")` is `["str", "1", "2", "3"]`. A method call is checked like a function call (argument count and types). Method names are not reserved: a function, a variable or a record field may share one, and `r.f(x)` on a record still calls the field. `s.length` is a property, so `s.length()` is an error. Widths and positions count bytes.

Builtins as methods
: `s.split(sep)`, `s.trim()` (also `s.strip()`), `s.index_of(sub)`, `s.last_index_of(sub)`, `s.ord()`, `s.byte_at(i)`, `s.tonum()` and `s.bytes()` are the builtins of the same name with `s` as the first argument.

`sep.join(xs)`
: `join(xs, sep)`: the receiver is the separator.

`s.trim_start()`, `s.trim_end()`
: Strip space, tab, CR and LF at one end only (also spelled `lstrip`, `rstrip`).

`s.pad_left(w, fill)`, `s.pad_right(w, fill)`, `s.center(w, fill)`
: Widen `s` to `w` bytes with `fill` (default `" "`, repeated and cut to fit); a longer `s` is returned unchanged. `center` puts the odd byte on the right.

`s.starts_with(p)`, `s.ends_with(p)`, `s.contains(p)`
: `bool`; `""` is a prefix, suffix and part of every string.

`s.count(p)`
: Non-overlapping occurrences, left to right (`0` for an empty `p`).

`s.replace(p, r)`
: Every non-overlapping occurrence of `p`, left to right, replaced by `r`; an empty `p` replaces nothing.

`s.upper()`, `s.lower()`
: ASCII case mapping; other bytes are kept.

`s.repeat(n)`
: `s` written `n` times; `""` when `n <= 0`.

`s.lines()`
: Split at each LF, dropping a CR just before it; a final newline does not start an empty line, so `"a\nb\n".lines()` is `["a", "b"]` and `"".lines()` is `[]`.

9.3 Conversions

`tostring(v)`
: String form: numbers as decimal (floats with six decimals), `true`/`false`, strings unchanged, options as their payload or `"none"`, arrays as `"[e0, e1, ...]"`, records as `"{k: v, ...}"`, maps as `"{"k": v, ...}"` (string keys quoted).

`tonum(s)` Parse a decimal integer (`0` on garbage).

`typeof(v)`
: The static type as a printable value (`typeof(none)` is `"none"`).

9.4 Format specifications

A field `{e:spec}` of an f-string (Section 2.4) formats `e` as Python's `format(e, spec)` would, in the same mini-language:

[[*fill*]*align*][*sign*][#][0][*width*][*grouping*][.*precision*][*type*]

align `<` left, `>` right, `^` centred (the odd byte goes right), `=` padding between the sign and the digits. Numbers align right by default, everything else left. A *fill* byte in front of it replaces the default space.

sign `+` on every number, `-` on the negative ones only (the default), or a space in front of the others.

`#` The prefix `0x`, `0X`, `0o` or `0b` for the radix types; a decimal point even with no digits after it for the float types.

`0` Pad with zeros after the sign: fill `0` and align `=`, unless the spec gives its own. The zeros are grouped too: `f"{255:#010_x}"` is `0x000_00ff`.

width The least number of bytes; padding makes up the rest.

grouping `,` or `_` between groups of three digits; `_` between groups of four for `x`, `X`, `o` and `b`, which do not take `,`.

precision Digits after the point for `f`, `F` and `%`, after the first digit for `e` and `E` (6 if omitted); for a string, the most bytes kept. The integer types take none.

type `d` decimal, `x`/`X` hexadecimal, `o` octal, `b` binary, `c` the byte `chr(e)`: integers only. `f`/`F` fixed point, `e`/`E` scientific, `%` times 100 in fixed point with a `%` sign: floats and integers. `s`: strings and the other non-numbers. Without a type, integers use `d`, floats `f` and everything else `s`.

Floats print exactly: the digits are those of the binary value, rounded half to even. So `f"{0.125:.2f}"` is `0.12`, and `f"{0.1:.20f}"` is `0.10000000000000000555`. Infinities and NaNs print as `inf` and `nan` (`INF` and `NAN` for `F` and `E`). A value that is neither a number nor a string — a `bool`, an array, a record, a map, an option — formats its `tostring` text as a string. Width and precision stop at 99999. A spec that does not read, or does not suit the field's static type, is a compile-time error; `f"{1.5:d}"` gets

```
format spec 'd':  type 'd' needs an integer, not float
```

Where Y differs from Python: a field without a spec is `tostring`, so a float shows six decimals (`f"{2.5}"` is `2.500000`); a float spec without a type means `f`, so `{x:.2}` gives two decimals rather than two significant digits; a `bool` is text, not a number; there are no `g`, `G` or `n` types and no nested fields; the fill is one byte, and widths count bytes.

9.5 Binary data

`bytes(s)` `[byte]` of the string's raw bytes.

`from_bytes(b)`
String from a `[byte]` or fixed `[byte; N]` array.

9.6 Options

`some(v)` A present option holding `v`.

`is_some(v)`
`true` unless `v` is `none`.

`unwrap(v)`
The payload; **traps** on `none`. The postfix form `v!` is the same operation.

9.7 Input/output and the environment

`print(x)` Print a value followed by a newline to stdout.

`puts(s)` Write a string to stdout, no newline added.

`eputs(s)` Same, to stderr (keeps `-time` output and program output apart).

`readline()`
One line from stdin without its newline, or `none` at end of input (`str?`). Stdin is buffered (16 KB per `read(2)`); before `exec()` starts a child, the unread part is given back when stdin is a file, so the child reads on from the next line. On a pipe, lines already buffered stay with the program, as with C stdio.

`clear_eof()`
No-op: `readline()` reads again whenever its buffer is empty, so end of input is never latched. Present so the same source runs on every compiler.

`argv()` `[str]` of the program's arguments.

`exit(code)`
Terminate with an exit code.

`clock_us()`
Monotonic microseconds as `long`: for intervals.

`time_us()`
Wall-clock microseconds since the Unix epoch (UTC) as `long`: the time of day, which jumps when the clock is set.

`heap_used()`
Bytes allocated so far.

sleep(ms)
: Suspend the process (`void`).

syscall(...)
: Raw Linux syscall wrapper; the first argument is the syscall number.

exec(cmd)
: Run a shell command, return its exit code. Removed from the binary entirely by `-no-exec` (then calling it is a compile error).

Files are the standard module `fs` (below), not builtins.

9.8 Files (the fs module)

`import fs;` brings the file system: whole files as text (`str`) or bytes (`[byte]`), paths, directories and file descriptors, all over raw syscalls. A read gives `none` and a write `false` when it fails; paths are relative to the working directory. A *mode* is the octal permission number (`420` is `0644`, `493` is `0755`); `-1` leaves a new file at `0644` minus the umask.

fs.read_file(path)
: The file as `str?`.

fs.read_bytes(path)
: The file as `[byte]?`.

fs.read_lines(path)
: Its lines, without their newlines, as `[str]?`.

fs.write_file(path, s, mode = -1)
: Create or truncate the file, write *s*, then (*mode* ≥ 0) set its permissions; `bool`.

fs.write_bytes(path, b, mode = -1)
: The same with a `[byte]`.

fs.append_file(path, s)
: Add *s* at the end (creating the file); `bool`.

fs.append_bytes(path, b)
: The same with a `[byte]`.

fs.copy_file(from, to)
: `bool`.

fs.rename(from, to)
: Rename or move; `bool`.

fs.remove_file(path)
: `bool`.

fs.exists(path)
: Whether there is such a path.

fs.is_file(path)
: A regular file (following a symlink).

fs.is_dir(path)
: A directory.

`fs.file_size(path)`
: Its size in bytes, `long?`.

`fs.set_mode(path, mode)`
: Set the permissions; `bool`.

`fs.make_dir(path, mode = 493)`
: `bool`.

`fs.make_dirs(path, mode = 493)`
: The directory and every one above it that is missing; `true` when it exists already.

`fs.list_dir(path)`
: The names in a directory, sorted, without `.` and `..`; `[str]?`.

`fs.remove_dir(path)`
: An empty directory; `bool`.

`fs.remove_all(path)`
: A file, or a directory and everything in it; `bool`.

`fs.cwd()` The working directory.

`fs.chdir(path)`
: Change it; `bool`.

`fs.open_file(path, how, mode = 420)`
: Open with a mode string: `"r"` read, `"w"` write (create, truncate), `"a"` append (create), `"x"` create a file that must not exist; a trailing `+` adds the other direction (`"r+"`, `"w+"`, `"a+"`). The file descriptor, or `-1`.

`fs.read_all(fd)`
: Everything left, as `str` (`fs.read_all_bytes(fd)`: as `[byte]`).

`fs.read_some(fd, n)`
: Up to *n* bytes (at most 65536); `[]` at the end.

`fs.write_text(fd, s)`
: `bool` (`fs.write_data(fd, b)`: a `[byte]`).

`fs.seek(fd, off, whence)`
: Move: from the start (0), the current position (1) or the end (2); the new position, or `-1`.

`fs.close_file(fd)`
: `bool`.

9.9 The web (the http, tls and net modules)

`import http;` is an HTTP/1.1 client, `https` included, in plain Y (it brings the modules below with it). One call makes a request; its options are a record whose fields may all be left out (Section 4.3):

```
import http;
let res = http.get("https://example.com/");
print(res.status, " ", res.headers["content-type"]);    // 200 text/html...
print(res.text());
let made = http.post("https://api.example.com/items", "{\"a\": 1}",
                     { headers: {"Content-Type": "application/json"} });
```

`get`, `head` and `delete` take the URL, `post`, `put` and `patch` the URL and a body, `request` a method and the URL; each then the options: `headers` (a map; they replace the client's own of the same name), `body`, `timeout` (seconds a read or a connect may wait, 30), `redirects` (how many to follow, 5), `insecure` (do not check an `https` server's chain) and `anchors` (PEM certificates to trust instead of the system's). The response is a record: `status` (0 when none came, and then `err` says why), `reason`, `headers` (names in lower case; a repeated header's values joined by `", "`, `set-cookie`'s by newlines), `size` (the Content-Length, or -1), `url` (after redirects), and the body, read as it arrives: `res.read(n)` gives up to *n* bytes (all the rest for $n \leq 0$, `[]` at its end), `res.text()` the rest as a `str`.

Many requests that look alike go through a session: a base URL their paths resolve against, headers for every request (`s.header(name, value)` adds one), connections kept open and used again, and the cookies servers set, sent back:

```
let s = http.session({ base: "https://api.example.com",
                       headers: {"Authorization": "Bearer t0ken"} });
let users = s.get("/users");
let made = s.post("/users", "{\"name\": \"ann\"}");
let gone = s.request("DELETE", "/users/7", { timeout: 5 });
s.close();
```

`import tls;` is the TLS 1.3 client underneath: `tls.connect(host, port)` (or `tls.connect_with(host, port, opts)`, with `anchors`, `alpn`, `insecure` and `timeout`) gives a connection whose `err` is `""` when the handshake held; `tls.send(c, bytes)`, `tls.recv(c)` (the next data, `none` at the end) and `tls.close(c)` use it. The server's chain must reach a trust anchor and name the host; X25519 and three suites are offered; there is no TLS 1.2. `import net;` has the sockets: `net.resolve(name)` (the IPv4 addresses, from `/etc/hosts` and DNS), `net.connect_tcp(host, port)`, `net.send_str`, `net.recv_str`, `net.close_fd`, and a small blocking server, `net.listen_http(port, handler)`.

9.10 Cryptography (the crypto and x509 modules)

`import crypto;` has what TLS uses, in plain Y over `[byte]`: `sha256`, `sha384`, `sha512`; `hmac` and `hkdf_extract`/`hkdf_expand` over `crypto.hash_sha256()` or `hash_sha384()`; `chacha_seal`/`chacha_open` (ChaCha20-Poly1305) and `gcm_seal`/`gcm_open` (AES-GCM), each `(key, nonce, aad, data)`, an open giving `none` for a forged message; `x25519` and `x25519_base`; `rsa_verify_pkcs1`, `rsa_verify_pss` and `ecdsa_verify` (over `crypto.p256()` or `p384()`); `random_bytes(n)`, `hex` and `unhex`. `import x509;` reads certificates: `x509.verify_chain(chain, host, now)` is `""` when the DER certificates *chain* (the server's first) reach a system trust anchor and name *host* at *now* (Unix seconds), else why not; `verify_chain_with` takes the anchors (`x509.anchors_pem(text)`), `x509.pem_certs(text)` splits a PEM file.

9.11 Pointers

`ptr_addr(p)`
: The address of a pointer as `long`.

Mutating and keying are symbolic, not functional: arrays use `+=` / `*=`, records use `r.f`, maps use `m['k']` and `m['k'] = v`. The legacy functions `push`, `prepend`, `map_get` and `map_set` do not exist.

10 Errors and diagnostics

10.1 Compile time

Static checking is fatal: a rejected program compiles to nothing. Diagnostics are one line each,

```
line:col:  error:  message
```

positioned in the user's source even when the compilation splices imported files or prelude/runtime code into it. An error in an imported file names that file, `path:line:col:  error:  message`, with the line counted in that file and the path as the import resolved it. The checker reports a root cause *once* (Section 4.14): after an error, the expression gets the error type, which causes no follow-on errors through the enclosing uses. Typical families:

- name errors: `undefined variable 'x'`;
- shape errors: `record has no field 'f'`;
- operator typing: `'+=' appended elements:  expected [long], got long`, `arrays support only '+=' and '++'`, `'++' needs a number target`;
- call errors: `call takes 2 args, got 1`;
- non-value use: `cannot index long (only arrays and maps)`;
- void use: `cannot bind a void value to 'x'`.

10.2 Warnings

A program that compiles can still draw *warnings*: code that is legal but looks like a mistake. They go to **stderr**, one line each, positioned like errors (an imported file's naming that file),

```
line:col:  warning:  message
```

and they never change the program: the binary is the same, byte for byte, with them or without, and `ycc -w` turns them off. The std units never warn; a package warns when it compiles. What draws one:

- a local binding nothing reads: `unused variable 'x'` (or `unused constant`, `unused function`), or one only assigned to: `variable 'x' is assigned but never read`;
- a value computed and dropped: `the value of 'x' is not used`, `the result of '+' is not used` (for `==` it adds `did you mean '='?`). A call's value may be dropped, and so may `ok && f()`;
- `unreachable code`: a statement after one that always returns;
- `'x' is assigned to itself` for `x = x`, and `'x < x' is always false` (or `>`; `x != x` is the NaN test and draws nothing);
- `division by zero`: `/`, `%`, `/=` or `%=` by a literal integer `0`;
- a key given twice in a record or map literal: `duplicate field 'a' in a record literal` (or `duplicate key`); the last one wins;
- a type that `type` could name once: a record of two or more fields, or a union, written out in full more than once (`the type {...}  is written out 2 times`) or where an alias names it already (`...  is already declared as 'Point'`); two aliases of one type (`type 'Spot' is the same type as 'Point'`); an alias declared twice (`type 'Tag' is declared more than once`), where the last declaration counts.

Parameters, loop variables, top-level bindings and names starting with `_` are never unused: `let _x = f();` keeps a value on purpose.

10.3 Run time

The compiled program traps — writes a one-line message to stdout and exits with status 1 — on:

condition	message
array index outside `[0, len)`	`runtime error: index out of bounds`
division or remainder by zero	`runtime error: division by zero`
shift by a negative count	`runtime error: negative shift count`
`unwrap(none)`, `none!`	`runtime error: unwrap of none`
slice assignment out of range	`runtime error: slice assignment out of bounds`
match with no arm covering a value	`runtime error: no match arm matched`
a union value cast to a member it does not hold (Section 4.4)	`runtime error: a union value cast to a member it does not hold`
memory exhausted	`runtime error: out of memory`
unbounded (non-tail) recursion	stack exhaustion: no limit, so the process dies on the guard page

These messages go to **stdout** (the trap stubs use `write(1, ...)` directly), so a program's real output stays recognizable; `eputs` is the way to send diagnostics to stderr.
Fixed arrays trap exactly like heap arrays; slices never trap (they clamp). Integer overflow is *not* an error (Section 5).

10.4 Exit status

A successfully run program exits with the code it chose (`exit(code)` or falling off the end, which is `0`); a trapped program exits with `1`. The compiler itself exits `0` on success (warnings or not), `1` on a rejected input.

10.5 Reproducibility

The compiler is deterministic: the same input produces byte-identical output on every run and in either implementation. This is checked continuously by comparing the two compilers' output binaries byte for byte, and by self-compilation (a compiler built by the compiler is identical to the one that built it).